TÓM TẮT

Trong bối cảnh các cuộc tấn công mạng ngày càng gia tăng về số lượng và mức độ tinh vi, việc giám sát và phản ứng sự cố an ninh đã trở thành một yêu cầu thiết yếu đối với mọi tổ chức. Tuy nhiên, các giải pháp Giám sát và Quản lý Sự kiện An ninh (SIEM) thương mại thường có chi phí cao, tạo ra rào cản cho nhiều doanh nghiệp. Đề tài này giải quyết thách thức đó bằng cách trình bày quá trình nghiên cứu, thiết kế và triển khai một hệ thống giám sát và phản ứng sự cố an ninh mạng toàn diện, hiệu quả về chi phí dựa trên nền tảng mã nguồn mở.

Mục tiêu cốt lõi của luận văn là xây dựng một quy trình an ninh đầu cuối, từ việc thu thập và chuẩn hóa log đa định dạng cho đến việc tự động phát hiện, cảnh báo và quản lý sự cố. Hệ thống được xây dựng trên bộ công cụ ELK Stack (Elasticsearch, Logstash, Kibana) kết hợp với Beats để tạo ra một đường ống dữ liệu mạnh mẽ. Để nâng cao khả năng tự động hóa, hệ thống tích hợp ElastAlert để thực thi các quy tắc phát hiện mối đe dọa theo thời gian thực và một giao diện quản lý sự cố (Case Management UI) chuyên dụng được phát triển bằng React, Node.js và MongoDB. Kiến trúc này cho phép tự động hóa luồng công việc từ khi một cảnh báo được kích hoạt cho đến khi một sự cố được tạo ra, giúp giảm thiểu thời gian phản ứng (MTTR).

Kết quả kiểm thử với các kịch bản giả lập như tấn công Brute-force, lạm dụng đặc quyền và đăng nhập đáng ngờ đã chứng minh hệ thống hoạt động hiệu quả, cung cấp cảnh báo giàu ngữ cảnh và hỗ trợ quá trình điều tra một cách nhanh chóng. Luận văn khẳng định tính khả thi của việc sử dụng công nghệ mã nguồn mở để xây dựng một giải pháp an ninh mạnh mẽ, linh hoạt và có khảibility, cung cấp một phương án giá trị cho các tổ chức mong muốn nâng cao năng lực phòng thủ của mình.

MỤC LỤC

DANH MỤC HÌNH ẢNH 5
DANH MỤC CÁC TỪ VIẾT TẮT 6
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 9
1.1. Bối cảnh và lý do chọn đề tài 9
1.2. Mục tiêu của đề tài 10
1.3. Phạm vi và các giả định 11
1.4. Cấu trúc của báo cáo 12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP LUẬN 12
2.1. Cơ sở lý thuyết 12
2.1.1. Tổng quan về hệ thống SIEM 12
2.1.2. Giới thiệu về bộ công cụ ELK Stack 13
2.1.3. Các khái niệm về an ninh mạng liên quan 13
2.2. Phương pháp luận 14
2.2.1. Quy trình phát hiện và phản ứng 9 giai đoạn 14
2.2.2. Nguyên tắc thiết kế quy tắc phát hiện 16
2.2.3. Hướng tiếp cận điều tra dựa trên trực quan hóa 16
2.2.4. Đo lường và cải tiến liên tục 17
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 17
3.1. Phân tích yêu cầu hệ thống 18
3.1.1. Yêu cầu chức năng 18
3.1.2. Yêu cầu phi chức năng 19
3.2. Thiết kế hệ thống 19
3.2.1. Thiết kế kiến trúc tổng thể 20
3.2.2. Phân vùng bảo mật và luồng dữ liệu 20
3.2.3. Thiết kế mô hình dữ liệu (Data Model) 21
3.2.4. Thiết kế hệ thống quản lý sự cố (Case Management) 21
CHƯƠNG 4: TRIỂN KHAI VÀ KIỂM THỬ HỆ THỐNG 23
4.1. Quy trình và các bước triển khai chi tiết 24

4.2. Phương pháp và Kịch bản kiểm thử.....25
4.2.1. Kiểm thử tích hợp (Integration Testing).....25
4.2.2. Kiểm thử toàn diện (End-to-End Testing).....26
4.2.3. Kịch bản Lạm dụng đặc quyền.....26
4.3. Triển khai hạ tầng và các thành phần.....27
CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ.....28
5.1. Kết quả đạt được.....28
5.1.2. Kết quả phát hiện và ví dụ về dòng thời gian sự cố.....30
5.2. Đánh giá hệ thống.....30
5.2.1. Ưu điểm.....30
5.2.2. Hạn chế.....31
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....31
6.1. Kết luận.....31
6.2. Hướng phát triển trong tương lai.....32

DANH MỤC HÌNH ẢNH

Hình 1: Các giai đoạn trong quy trình phản ứng sự cố 15
Hình 2: Sơ đồ logic của quá trình phát hiện mối đe dọa 16
Hình 3: Bảng điều khiển giám sát thời gian thực hỗ trợ điều tra 17
Hình 4: Vòng đời và giao diện mẫu của hệ thống quản lý sự cố (Case Management). 22
Hình 5: Ví dụ về các biểu đồ phân tích chi tiết trong một bảng điều khiển 28
Hình 6: Giao diện Bảng điều khiển chính của Hệ thống Quản lý Sự cố (Case Management UI) 29

DANH MỤC CÁC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
SIEM	Security Information and Event Management	Quản lý Sự kiện và Thông tin An ninh
ELK Stack	Elasticsearch, Logstash, Kibana	Bộ công cụ Elasticsearch, Logstash, Kibana
SOC	Security Operations Center	Trung tâm Điều hành An ninh mạng
MTTD	Mean Time To Detect	Thời gian Trung bình để Phát hiện
MTTR	Mean Time To Respond	Thời gian Trung bình để Phản ứng
UI	User Interface	Giao diện người dùng
API	Application Programming Interface	Giao diện Lập trình Ứng dụng
TLS	Transport Layer Security	Bảo mật Tầng truyền tải
GeoIP	Geolocation by IP address	Định vị địa lý qua địa chỉ IP
RBAC	Role-Based Access Control	Kiểm soát Truy cập dựa trên Vai trò
ILM	Index Lifecycle Management	Quản lý Vòng đời Chỉ mục (của Elasticsearch)
KPI	Key Performance Indicator	Chỉ số Hiệu suất Chính
SOAR	Security Orchestration, Automation, and Response	Điều phối, Tự động hóa và Phản ứng An ninh
SSO	Single Sign-On	Đăng nhập một lần
PII	Personally Identifiable Information	Thông tin Nhận dạng Cá nhân
NTP	Network Time Protocol	Giao thức Đồng bộ Thời gian Mạng

LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa, dữ liệu đã trở thành tài sản vô giá đối với mọi tổ chức và doanh nghiệp. Đi cùng với sự phát triển đó là sự gia tăng không ngừng của các mối đe dọa an ninh mạng, với mức độ ngày càng tinh vi và phức tạp. Để bảo vệ tài sản thông tin, các tổ chức phải đối mặt với thách thức to lớn trong việc giám sát một khối lượng khổng lồ dữ liệu nhật ký (log) phát sinh từ vô số hệ thống khác nhau. Tuy nhiên, sự phân mảnh, thiếu đồng nhất về định dạng và áp lực phải phát hiện, phản ứng gần như tức thời đã khiến cho công tác đảm bảo an ninh trở nên khó khăn hơn bao giờ hết.

Các giải pháp Giám sát và Quản lý Sự kiện An ninh (SIEM) thương mại tuy mạnh mẽ nhưng thường đi kèm với chi phí bản quyền đắt đỏ, tạo ra rào cản lớn cho nhiều doanh nghiệp, đặc biệt là các đơn vị có ngân sách hạn chế. Xuất phát từ thực tiễn đó, nhu cầu về một giải pháp hiệu quả, linh hoạt, có khả năng tùy chỉnh cao và chi phí hợp lý đã trở nên vô cùng cấp thiết.

Với mong muốn giải quyết bài toán trên, tôi đã thực hiện đề tài: “Xây dựng Bảng điều khiển Giám sát và Phản ứng Sự cố An ninh mạng dựa trên nền tảng ELK Stack”. Đề tài này tập trung vào việc nghiên cứu và triển khai một hệ thống hoàn chỉnh, hợp nhất toàn bộ quy trình từ thu thập, chuẩn hóa, phân tích dữ liệu đến phát hiện, cảnh báo và hỗ trợ phản ứng sự cố. Bằng cách tận dụng sức mạnh của bộ công cụ mã nguồn mở ELK Stack (Elasticsearch, Logstash, Kibana) và các công nghệ phụ trợ, giải pháp không chỉ giúp tự động hóa và rút ngắn thời gian xử lý sự cố (MTTD/MTTR) mà còn xây dựng một quy trình có khả năng kiểm toán minh bạch, phục vụ cho việc cải tiến liên tục.

Báo cáo này sẽ trình bày chi tiết toàn bộ quá trình thực hiện đề tài, từ việc đặt nền móng lý thuyết, phân tích và thiết kế kiến trúc hệ thống, cho đến các bước triển khai cụ thể và đánh giá kết quả đạt được. Qua đó, tác giả hy vọng đề tài sẽ mang lại một cái nhìn thực tiễn và một giải pháp khả thi, giúp các tổ chức có thể tự xây dựng và vận hành một hệ thống giám sát an ninh hiệu quả, góp phần nâng cao năng lực phòng thủ trong bối cảnh an ninh mạng đầy biến động hiện nay.

Mặc dù đã có nhiều cố gắng, nhưng do kiến thức và thời gian có hạn, báo cáo chắc chắn không thể tránh khỏi những thiếu sót. Kính mong nhận được sự chỉ dẫn, góp ý của Quý Thầy Cô và các bạn để đề tài được hoàn thiện hơn.

Xin trân trọng cảm ơn

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

Chương này trình bày tổng quan về bối cảnh nghiên cứu, xác định mục tiêu, phạm vi và cấu trúc của báo cáo. Mục tiêu chính là đặt nền móng cho việc xây dựng một hệ thống giám sát và phản ứng sự cố an ninh mạng hiệu quả.

1.1. Bối cảnh và lý do chọn đề tài

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ, các tổ chức và doanh nghiệp phải đối mặt với một khối lượng dữ liệu an ninh khổng lồ. Dữ liệu này phát sinh liên tục từ vô số nguồn khác nhau như hệ điều hành, ứng dụng, cơ sở dữ liệu, và các thiết bị mạng. Tuy nhiên, việc khai thác giá trị từ dữ liệu này gặp rất nhiều thách thức.

Thứ nhất, sự phân mảnh và thiếu đồng nhất của dữ liệu là rào cản lớn nhất. Log từ các hệ thống khác nhau (như journald, syslog, auditd của Linux, log ứng dụng, proxy) thường có cấu trúc, múi giờ và quy ước đặt tên không đồng nhất. Điều này khiến cho việc điều tra thủ công trở nên chậm chạp, khó tái lập, và các quy tắc phát hiện dễ bị "vỡ" khi có sự thay đổi từ nguồn.

Thứ hai, áp lực phát hiện và phản ứng trong thời gian gần thực ngày càng gia tăng. Các mối đe dọa tinh vi như tấn công Brute-force, đăng nhập từ các vị trí địa lý bất thường, hay lạm dụng đặc quyền có thể gây ra thiệt hại nghiêm trọng chỉ trong vài phút. Nếu không có một quy trình xử lý được chuẩn hóa và tự động hóa, thời gian trung bình để phát hiện (Mean Time To Detect - MTTD) và thời gian trung bình để phản ứng (Mean Time To Respond - MTTR) sẽ kéo dài, làm tăng rủi ro về tài chính và uy tín.

Cuối cùng, nhu cầu về một quy trình chuẩn hóa đầu cuối là vô cùng cấp thiết. Một hệ thống hiệu quả phải có khả năng biến log thô thành những thông tin chi tiết (insights) có giá trị, cung cấp giao diện trực quan để dẫn dắt quá trình điều tra, gửi cảnh báo chứa đầy đủ ngữ cảnh và liên kết "một chạm" (one-click) đến bằng chứng, đồng thời trang bị một giao diện quản lý sự cố (case management) để đảm bảo tính minh bạch và khả năng kiểm toán.

Xuất phát từ những thách thức trên, đề tài "Xây dựng Bảng điều khiển Giám sát và Phản ứng Sự cố An ninh mạng dựa trên nền tảng ELK Stack" được thực hiện nhằm giải quyết bài toán hợp nhất và tự động hóa quy trình an ninh, đặc biệt là trong các môi trường có ngân sách hạn chế nhưng yêu cầu cao về khả năng tùy chỉnh và mở rộng.

1.2. Mục tiêu của đề tài

Đề tài tập trung vào việc thực hiện các mục tiêu chính sau:

- Về dữ liệu: Xây dựng một quy trình (pipeline) thu thập log nhất quán, có khả năng đệm và chịu lỗi. Chuẩn hóa và làm giàu dữ liệu một cách tự động (sử dụng các kỹ thuật như grok, geoip, fingerprint) để tạo ra một nguồn dữ liệu sạch, đáng tin cậy, phục vụ cho việc phân tích và điều tra chuyên sâu.
- Về phát hiện và cảnh báo: Thiết kế các quy tắc phát hiện linh hoạt dựa trên nguyên tắc "trường + cửa sổ thời gian" cho các kịch bản tấn công phổ biến (Brute-force, đăng nhập đáng ngờ, lạm dụng đặc quyền). Các cảnh báo được tạo ra phải giàu ngữ cảnh, chứa các liên kết sâu (deep link) đến bằng chứng trên giao diện trực quan, giúp rút ngắn thời gian điều tra.
- Về phản ứng và kiểm toán: Xây dựng một giao diện quản lý sự cố (Case UI) chuyên dụng để ghi lại dòng thời gian (timeline), chỉ định người chịu trách nhiệm, lưu trữ bằng chứng và theo dõi các chỉ số hiệu suất quan trọng (KPIs) như MTTD/MTTR. Điều này giúp chuẩn hóa quy trình phản ứng và phục vụ cho việc kiểm toán sau này.
- Về triển khai và chi phí: Ưu tiên sử dụng các công nghệ mã nguồn mở, cụ thể là bộ công cụ ELK Stack (Elasticsearch, Logstash, Kibana) và triển khai trên nền tảng Docker/Ubuntu để tối ưu hóa chi phí và đảm bảo khả năng mở rộng. Toàn bộ hệ thống được cấu hình như mã (Configuration-as-Code) và tuân thủ các nguyên tắc bảo mật như mã hóa đầu cuối (TLS end-to-end) và phân quyền tối thiểu (RBAC).

1.3. Phạm vi và các giả định

Phạm vi của đề tài:

- Nguồn log: Tập trung vào việc thu thập và phân tích log từ hệ điều hành Linux (bao gồm syslog, auth.log, và auditd). Có thể mở rộng để thu thập log từ các nguồn mạng như proxy hoặc DNS.
- Công nghệ: Sử dụng bộ ba ELK Stack cho các tác vụ chính, kết hợp với Beats (Filebeat, Auditbeat) để thu thập log. Giao diện quản lý sự cố được xây dựng bằng React, Node.js và MongoDB. Hệ thống cảnh báo được tích hợp với các kênh như Email, Slack, và Telegram.

- Kịch bản phát hiện: Tập trung vào ba kịch bản chính: tấn công Brute-force, đăng nhập từ vị trí địa lý đáng ngờ, và lạm dụng đặc quyền.

Các giả định:

- Hạ tầng Docker và hệ điều hành Ubuntu đã được chuẩn bị sẵn sàng.
- Các nguồn log đáng tin cậy và được đồng bộ hóa thời gian bằng giao thức NTP (Network Time Protocol).
- Các cổng kết nối (connector) đến dịch vụ Email, Slack, Telegram đã sẵn sàng để tích hợp.
- Việc truy cập vào các giao diện của Elasticsearch và Kibana được thực hiện qua giao thức HTTPS để đảm bảo an toàn.

1.4. Cấu trúc của báo cáo

Ngoài Lời mở đầu và Kết luận, báo cáo được tổ chức thành 4 chương chính:

- Chương 1 - Tổng quan về đề tài: Giới thiệu bối cảnh, lý do, mục tiêu, phạm vi và cấu trúc của báo cáo.
- Chương 2 - Cơ sở lý thuyết và Phương pháp luận: Trình bày các kiến thức nền tảng về SIEM, ELK Stack và phương pháp tiếp cận được sử dụng trong đề tài.
- Chương 3 - Phân tích và Thiết kế hệ thống: Đi sâu vào việc phân tích các yêu cầu chức năng, phi chức năng và thiết kế chi tiết kiến trúc hệ thống.
- Chương 4 - Triển khai hệ thống: Mô tả quá trình triển khai thực tế các thành phần, từ công cụ, quy tắc phát hiện đến hạ tầng.
- Chương 5 - Kết quả và Đánh giá: Trình bày các kết quả đạt được qua các giao diện, cảnh báo mẫu và đưa ra đánh giá về các ưu, nhược điểm của hệ thống.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ PHƯƠNG PHÁP LUẬN

Chương này trình bày các khái niệm lý thuyết nền tảng về hệ thống Giám sát và Quản lý sự kiện an ninh (SIEM), giới thiệu chi tiết về các thành phần của bộ công cụ Elastic Stack

(ELK), và làm rõ phương pháp luận được áp dụng để xây dựng hệ thống trong khuôn khổ của đề tài.

2.1. Cơ sở lý thuyết

2.1.1. Tổng quan về hệ thống SIEM

Hệ thống Giám sát và Quản lý sự kiện an ninh (Security Information and Event Management - SIEM) là một giải pháp công nghệ có chức năng thu thập, tổng hợp, và phân tích dữ liệu nhật ký (log) từ nhiều nguồn khác nhau trong hạ tầng công nghệ thông tin của một tổ chức. Mục tiêu chính của SIEM là cung cấp một cái nhìn toàn diện về trạng thái an ninh, giúp phát hiện các mối đe dọa, điều tra sự cố và đáp ứng các yêu cầu về tuân thủ. Một hệ thống SIEM hiện đại thường bao gồm các khả năng cốt lõi như: thu thập log, chuẩn hóa dữ liệu, tương quan sự kiện, cảnh báo, và tạo báo cáo.

2.1.2. Giới thiệu về bộ công cụ ELK Stack

ELK Stack (viết tắt của Elasticsearch, Logstash, Kibana) là một bộ ba công cụ mã nguồn mở mạnh mẽ, thường được sử dụng để xây dựng các giải pháp tìm kiếm, phân tích và trực quan hóa dữ liệu lớn, bao gồm cả các hệ thống SIEM.

- Elasticsearch: Là trái tim của bộ công cụ, đóng vai trò là một công cụ tìm kiếm và phân tích phân tán. Dựa trên Apache Lucene, Elasticsearch cung cấp khả năng lưu trữ, lập chỉ mục (indexing) và truy vấn dữ liệu với tốc độ rất nhanh, phù hợp cho việc phân tích dữ liệu log theo thời gian thực.
- Logstash: Là thành phần chịu trách nhiệm thu thập, xử lý và chuyển tiếp dữ liệu. Logstash có thể nhận dữ liệu từ nhiều nguồn, thực hiện các biến đổi phức tạp như phân tích cú pháp (parsing), làm giàu (enrichment) và chuẩn hóa (normalization) trước khi gửi đến một "kho" lưu trữ như Elasticsearch.
- Kibana: Là giao diện người dùng, cho phép người dùng trực quan hóa dữ liệu được lưu trữ trong Elasticsearch. Với Kibana, người dùng có thể tạo các bảng điều khiển (dashboards), biểu đồ, bản đồ và thực hiện các truy vấn tương tác để khám phá dữ liệu, từ đó tìm ra các thông tin chi tiết có giá trị.

- Beats: Là một tập hợp các agent thu thập dữ liệu gọn nhẹ, được cài đặt trên các máy chủ nguồn để gửi các loại dữ liệu cụ thể về cho Logstash hoặc Elasticsearch. Trong phạm vi đề tài, các Beats được sử dụng bao gồm Filebeat (thu thập file log) và Auditbeat (thu thập log kiểm toán của Linux).

2.1.3. Các khái niệm về an ninh mạng liên quan

- Tấn công Brute-force: Là kỹ thuật thử lặp đi lặp lại nhiều tên người dùng và mật khẩu khác nhau cho đến khi tìm ra thông tin đăng nhập chính xác.
- Lạm dụng đặc quyền (Privilege Misuse): Là hành vi của một người dùng hợp lệ nhưng sử dụng quyền truy cập của mình để thực hiện các hành động không được phép.
- MTTD (Mean Time To Detect): Thời gian trung bình để phát hiện một sự cố an ninh kể từ khi nó xảy ra.
- MTTR (Mean Time To Respond): Thời gian trung bình để khắc phục một sự cố an ninh sau khi nó được phát hiện.

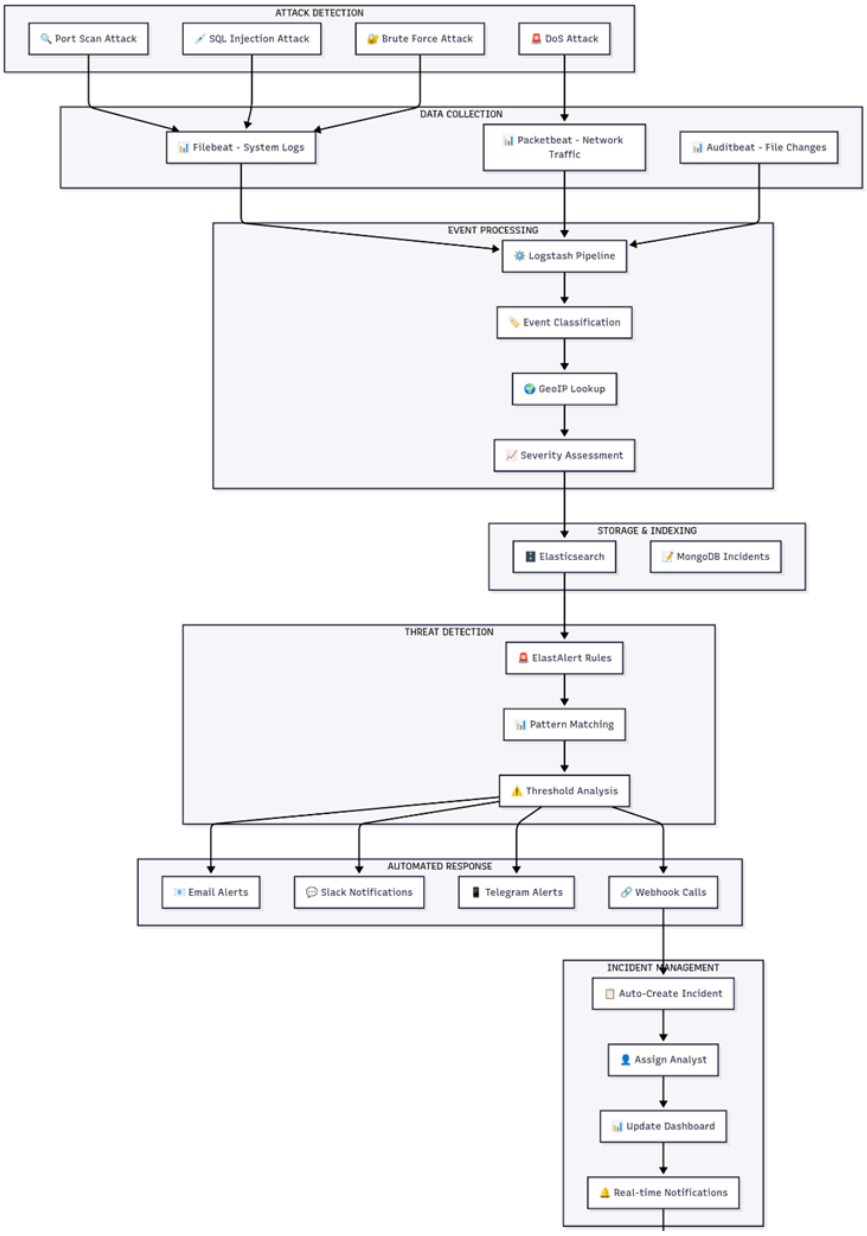
2.2. Phương pháp luận

Để chuyển hóa log thô thành các tín hiệu phát hiện đáng tin cậy và xây dựng một quy trình phản ứng có kiểm toán, đề tài áp dụng một phương pháp luận tổng thể, được thể hiện qua các khía cạnh sau:

2.2.1. Quy trình phát hiện và phản ứng 9 giai đoạn

1. Attack (Tấn công): Một sự kiện an ninh thực tế hoặc giả lập xảy ra.
2. Device Logging (Ghi log tại thiết bị): Hệ điều hành và dịch vụ ghi nhận sự kiện vào các file log.
3. Collection (Thu thập): Beats thu thập log và gửi về trung tâm.
4. Preprocessing (Tiền xử lý): Logstash phân tích, chuẩn hóa và làm giàu dữ liệu.
5. Storage (Lưu trữ): Elasticsearch lập chỉ mục và lưu trữ dữ liệu đã xử lý.
6. Analysis (Phân tích): Chuyên viên sử dụng Kibana để truy vấn và trực quan hóa dữ liệu.
7. Detection (Phát hiện): Các quy tắc trong ElastAlert/Watcher quét dữ liệu trên Elasticsearch để tìm các mẫu bất thường.

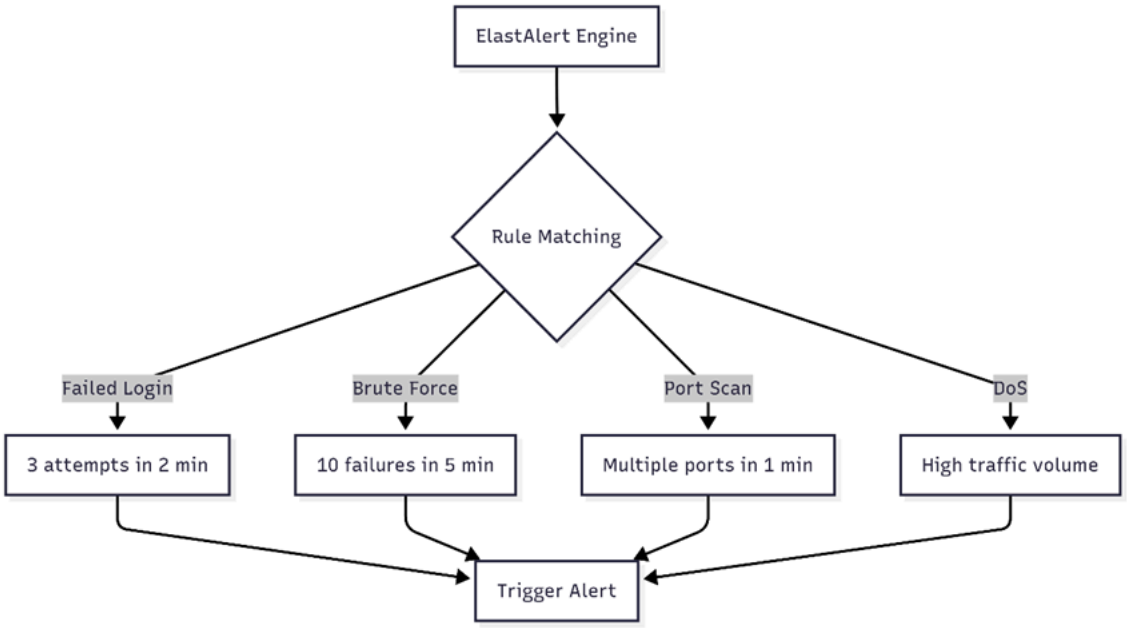
8. Alerting (Cảnh báo): Hệ thống tự động gửi cảnh báo qua các kênh đã định cấu hình.
9. Response & Case (Phản ứng và Quản lý sự cố): Chuyên viên tiếp nhận cảnh báo, tạo và xử lý sự cố trong Case UI.



Hình 1: Các giai đoạn trong quy trình phản ứng sự cố

2.2.2. Nguyên tắc thiết kế quy tắc phát hiện

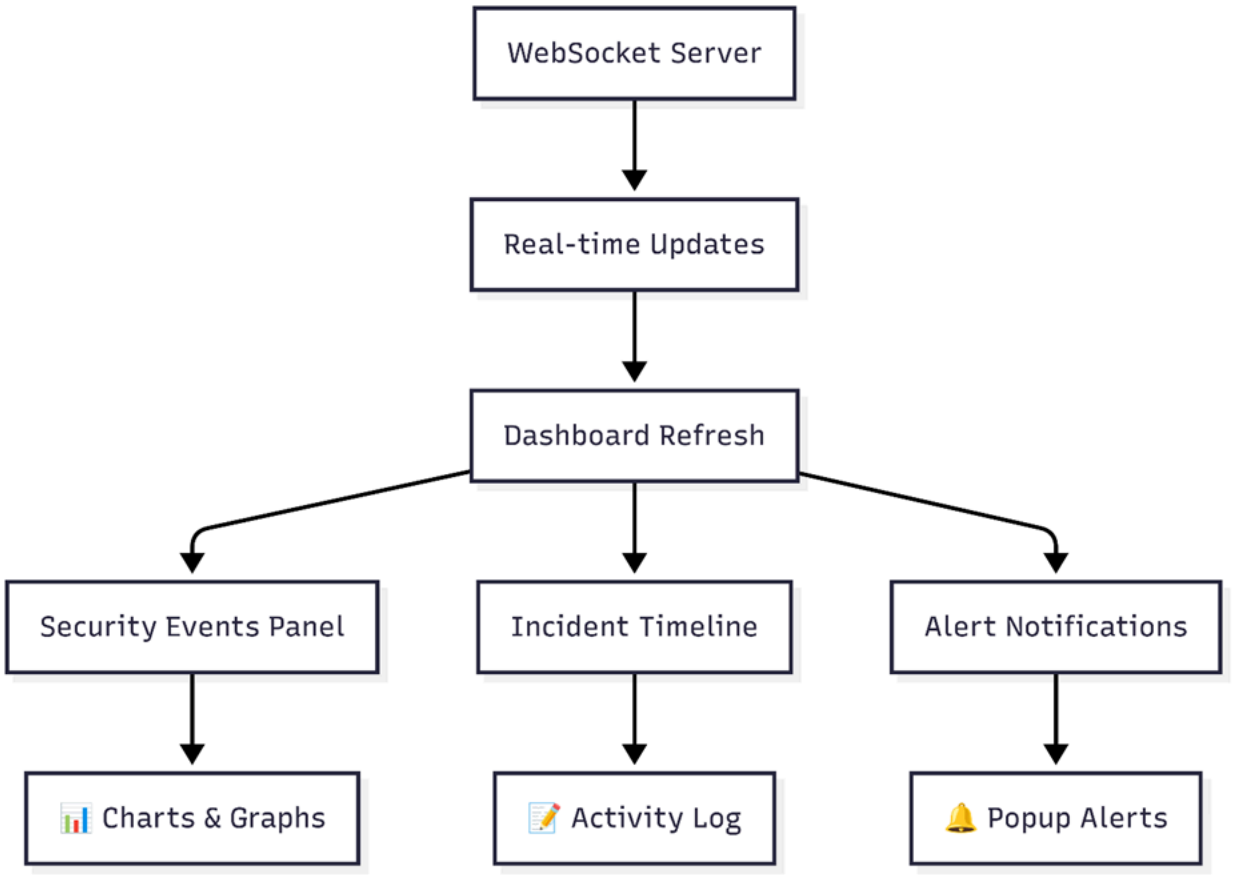
Các quy tắc phát hiện được xây dựng dựa trên nguyên tắc cốt lõi "trường + cửa sổ thời gian". Thay vì phụ thuộc vào các chuỗi văn bản tự do dễ thay đổi, các quy tắc sẽ dựa trên các trường dữ liệu đã được chuẩn hóa (ví dụ: user.name, source.ip, outcome). Việc kết hợp các trường này với một cửa sổ thời gian (ví dụ: "30 phút") và một ngưỡng (ví dụ: "≥ 20 lần") cho phép tạo ra các quy tắc phát hiện hành vi rất mạnh mẽ và bền vững.



Hình 2: Sơ đồ logic của quá trình phát hiện mối đe dọa

2.2.3. Hướng tiếp cận điều tra dựa trên trực quan hóa

Phương pháp luận của đề tài khuyến khích cách tiếp cận "điều tra dựa trên trực quan hóa" (visualization-first investigation). Thay vì đọc qua hàng ngàn dòng log, chuyên viên phân tích sẽ bắt đầu từ các bảng điều khiển tổng quan trên Kibana. Từ các biểu đồ, họ có thể nhanh chóng xác định các đỉnh bất thường (spikes), các thực thể nổi bật (top users, top IPs), sau đó thực hiện "khoan" (drilldown) sâu hơn vào các dữ liệu chi tiết liên quan để tìm ra bằng chứng, giúp rút ngắn đáng kể thời gian điều tra.



Hình 3: Bảng điều khiển giám sát thời gian thực hỗ trợ điều tra

2.2.4. Đo lường và cải tiến liên tục

Hệ thống được thiết kế để có thể đo lường hiệu quả thông qua các chỉ số như: độ chính xác của quy tắc, độ trễ của cảnh báo (alert latency), và thời gian để tìm ra bằng chứng đầu tiên. Dựa trên các số liệu này, các quy tắc, ngưỡng và danh sách loại trừ (allowlist) sẽ được tinh chỉnh liên tục theo chu kỳ để nâng cao hiệu quả hoạt động của hệ thống.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này đi sâu vào việc phân tích các yêu cầu cụ thể của hệ thống, từ đó đề xuất một kiến trúc và thiết kế chi tiết. Mục tiêu là đảm bảo hệ thống không chỉ đáp ứng được các

nhuorescent chức năng và phi chức năng đã đề ra mà còn có khả năng mở rộng, bảo mật và dễ dàng bảo trì.

3.1. Phân tích yêu cầu hệ thống

Việc xác định rõ các yêu cầu là bước nền tảng để đảm bảo hệ thống được xây dựng đúng mục đích và có thể kiểm chứng được. Các yêu cầu được chia thành hai nhóm chính: chức năng và phi chức năng.

3.1.1. Yêu cầu chức năng

- Thu thập log (Ingestion): Hệ thống phải có khả năng tiếp nhận log một cách bền bỉ từ nhiều nguồn khác nhau trên các máy chủ Linux, cụ thể là các file log hệ thống (syslog), log xác thực (auth.log) và log kiểm toán (auditd). Quá trình thu thập phải đảm bảo không mất mát dữ liệu ngay cả khi có sự cố kết nối tạm thời, thông qua việc sử dụng hàng đợi (queue) và cơ chế thử lại (retry).
- Xử lý và làm giàu log (Processing & Enrichment): Dữ liệu log thô sau khi thu thập phải được chuẩn hóa và làm giàu. Logstash sẽ thực hiện phân tích cú pháp để trích xuất các trường thông tin quan trọng, chuẩn hóa kiểu dữ liệu (ngày tháng, số, địa chỉ IP), và làm giàu thêm các thông tin ngữ cảnh như vị trí địa lý dựa trên IP (GeoIP) và thông tin của máy chủ.
- Lưu trữ và tìm kiếm (Storage & Search): Dữ liệu đã xử lý sẽ được lưu trữ trong Elasticsearch. Hệ thống phải hỗ trợ việc lập chỉ mục (indexing) dữ liệu theo ngày để tối ưu quản lý và truy vấn. Người dùng phải có khả năng tìm kiếm và lọc log theo các trường quan trọng như host.name, user.name, source.ip trong các khoảng thời gian phổ biến (24 giờ, 7 ngày).
- Trực quan hóa (Visualization): Kibana phải cung cấp các bảng điều khiển (dashboards) được thiết kế riêng cho từng mục đích phân tích như Xác thực (Authentication) và Kiểm toán (Audit). Các dashboard này phải hiển thị thông tin một cách trực quan, giúp người dùng dễ dàng phát hiện các xu hướng và điểm bất thường.
- Phát hiện và Cảnh báo (Detection & Alerting): Hệ thống phải có khả năng tự động phát hiện các hành vi tấn công dựa trên các quy tắc đã định nghĩa trước, tập trung vào 3 kịch bản chính: tấn công dò mật khẩu (Brute-force), đăng nhập từ vị trí địa lý đáng ngờ (Suspicious GeoIP), và lạm dụng đặc quyền (Privilege Misuse). Khi

phát hiện, hệ thống phải gửi cảnh báo ngay lập tức qua các kênh như Email, Slack, hoặc Telegram.

- Quản lý sự cố (Incident Management): Phải có một giao diện (Case UI) để quản lý vòng đời của một sự cố, từ khi nó được tạo ra cho đến khi được giải quyết. Giao diện này cho phép ghi nhận lại quá trình điều tra, gán người xử lý, và theo dõi các chỉ số đo lường hiệu quả (KPIs).

3.1.2. Yêu cầu phi chức năng

Đây là các yêu cầu về đặc tính vận hành của hệ thống, định nghĩa các tiêu chuẩn về hiệu năng, bảo mật, và độ tin cậy.

- Hiệu năng (Performance): Độ trễ từ khi log được ghi nhận tại nguồn cho đến khi cảnh báo được gửi đi (P95 alert latency) không được vượt quá 5 phút. Các dashboard tổng quan trên Kibana phải có thời gian phản hồi dưới 10 giây cho các truy vấn trong khoảng 24 giờ.
- Độ tin cậy (Reliability): Các thành phần của hệ thống (được triển khai dưới dạng container) phải có cơ chế tự động khởi động lại khi gặp lỗi (health checks). Hệ thống phải có cơ chế sao lưu (snapshot) định kỳ cho dữ liệu và cấu hình để có thể khôi phục khi cần thiết.
- Bảo mật (Security): Mọi giao tiếp dữ liệu giữa các thành phần của hệ thống (Beats, Logstash, Elasticsearch, Kibana) phải được mã hóa đầu cuối bằng giao thức TLS. Hệ thống phải áp dụng cơ chế phân quyền dựa trên vai trò (RBAC) để đảm bảo người dùng chỉ có thể truy cập vào các chức năng và dữ liệu được phép.
- Khả năng bảo trì (Maintainability): Toàn bộ cấu hình của hệ thống phải được quản lý dưới dạng mã nguồn (Infrastructure as Code), cho phép việc triển khai, cập nhật và quay lui (rollback) được thực hiện một cách nhất quán và tự động.
- Tuân thủ (Compliance): Hệ thống phải có chính sách lưu trữ và xóa dữ liệu tự động (ILM - Index Lifecycle Management) để tuân thủ các quy định về lưu trữ. Các thông tin nhạy cảm (PII) cần có khả năng được che mờ (masking) khi cần thiết.

3.2. Thiết kế hệ thống

Dựa trên các yêu cầu đã phân tích, một kiến trúc hệ thống theo lớp được đề xuất để đảm bảo tính module hóa, linh hoạt và bảo mật.

3.2.1. Thiết kế kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc nhiều lớp, mỗi lớp đảm nhiệm một vai trò chuyên biệt, giúp dễ dàng quản lý và mở rộng.

- Lớp Thu thập (Ingest Layer): Bao gồm các agent Beats (Filebeat, Auditbeat) được cài đặt trên máy chủ nguồn, có nhiệm vụ thu thập và đẩy log về Logstash một cách an toàn qua kênh mã hóa TLS.
- Lớp Xử lý (Processing Layer): Logstash đóng vai trò trung tâm, nhận log từ Beats, thực hiện phân tích, làm giàu (gắn thông tin GeoIP) và chuẩn hóa dữ liệu theo một cấu trúc chung.
- Lớp Lưu trữ & Tìm kiếm (Storage & Search Layer): Elasticsearch nhận dữ liệu đã sạch từ Logstash, tiến hành lập chỉ mục để phục vụ cho việc tìm kiếm, phân tích và phát hiện mối đe dọa.
- Lớp Trình bày & Tương tác (Presentation & Interaction Layer): Bao gồm Kibana và Case UI. Kibana dùng để trực quan hóa và điều tra dữ liệu. Case UI (một ứng dụng React/NodeJS) dùng để quản lý quy trình phản ứng sự cố.
- Lớp Phát hiện & Cảnh báo (Detection & Alerting Layer): Sử dụng ElastAlert hoặc Watcher của Elasticsearch để liên tục truy vấn dữ liệu, áp dụng các quy tắc phát hiện và kích hoạt cảnh báo khi có sự kiện trùng khớp.

3.2.2. Phân vùng bảo mật và luồng dữ liệu

Để tăng cường an ninh, hệ thống được chia thành các vùng mạng logic (security zones), mỗi vùng có một chức năng và chính sách truy cập riêng.

- Vùng Thu thập (Collector Zone): Là các máy chủ nguồn cài đặt Beats.
- Vùng Xử lý (Processing Zone): Chứa Logstash, đóng vai trò là một DMZ, chỉ nhận kết nối từ các Beats.
- Vùng Dữ liệu (Data Zone): Chứa cụm Elasticsearch và Kibana, là vùng được bảo vệ nghiêm ngặt nhất.
- Vùng Ứng dụng (Application Zone): Chứa ứng dụng Case UI và cơ sở dữ liệu MongoDB, tách biệt với vùng dữ liệu.
- Luồng dữ liệu được thiết kế theo một chiều và được mã hóa tại mọi điểm: Beats → Logstash → Elasticsearch. Kibana và Case UI truy vấn dữ liệu từ Elasticsearch, trong khi hệ thống cảnh báo sẽ gửi thông báo ra các kênh bên ngoài (Email/Slack).

3.2.3. Thiết kế mô hình dữ liệu (Data Model)

Một mô hình dữ liệu nhất quán là yếu tố quyết định sự thành công của hệ thống. Đề tài áp dụng một lược đồ (schema) chung cho các loại log khác nhau, với các trường trọng yếu được quy ước như sau:

- @timestamp: Thời gian xảy ra sự kiện (bắt buộc).
- host.name: Tên máy chủ nguồn.
- source.ip: Địa chỉ IP nguồn của sự kiện.
- user.name: Tên người dùng liên quan.
- event.module: Loại log (ví dụ: auth, audit).
- event.action: Hành động cụ thể (ví dụ: accepted_password, user_login).
- outcome: Kết quả của hành động (success hoặc failure).
- geoip.country_name: Tên quốc gia được làm giàu từ IP nguồn.
- geoip.location: Tọa độ địa lý.

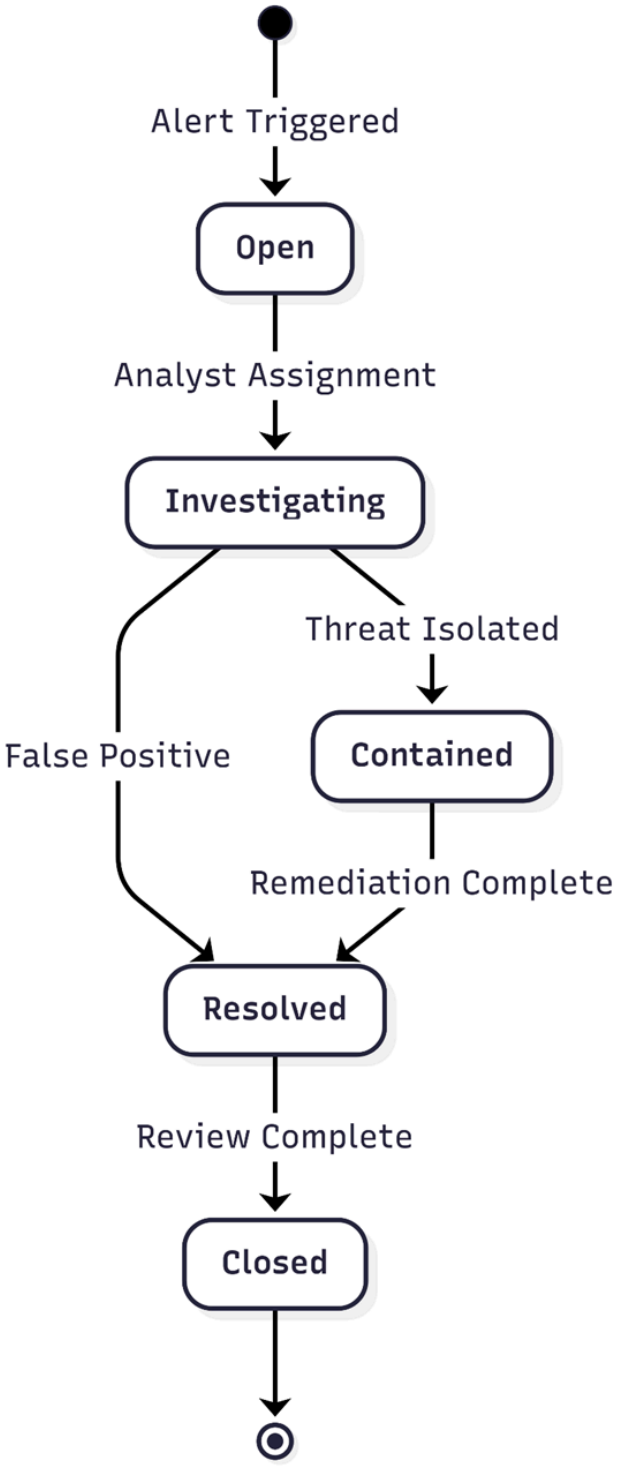
Việc chuẩn hóa này giúp cho các truy vấn, dashboard và quy tắc phát hiện có thể được xây dựng một cách đồng nhất và bền vững.

3.2.4. Thiết kế hệ thống quản lý sự cố (Case Management)

Hệ thống quản lý sự cố (Case UI) được thiết kế như một ứng dụng web độc lập, tương tác với các thành phần khác qua API.

- Cấu trúc dữ liệu (Schema): Mỗi sự cố (case) sẽ được lưu trong MongoDB với các trường thông tin chính: case_id, title, severity (mức độ nghiêm trọng), status (trạng thái: Open, Investigating, Contained, Resolved, Closed), owner (người xử lý), và một timeline (dòng thời gian).
- Dòng thời gian (Timeline): Đây là thành phần quan trọng nhất, ghi lại mọi hành động liên quan đến sự cố, bao gồm: thời gian, người thực hiện, hành động, và ghi chú. Điều này đảm bảo toàn bộ quá trình xử lý được kiểm toán một cách minh bạch.
- API: Hệ thống cung cấp các endpoint API để:
    - Tự động tạo case mới khi có cảnh báo từ ElastAlert.
    - Cho phép chuyên viên cập nhật trạng thái, gán người xử lý, và thêm các ghi chú điều tra vào timeline.

- Lấy danh sách các case để hiển thị trên giao diện.



Hình 4: Vòng đời và giao diện mẫu của hệ thống quản lý sự cố (Case Management).

CHƯƠNG 4: TRIỂN KHAI VÀ KIỂM THỬ HỆ THỐNG

Chương này trình bày chi tiết về quy trình triển khai thực tế các thành phần của hệ thống đã được thiết kế ở Chương 3, đồng thời mô tả các phương pháp và kịch bản kiểm thử được thực hiện để đảm bảo hệ thống hoạt động chính xác, ổn định và đáp ứng đúng các yêu cầu đã đề ra.

Để hiện thực hóa kiến trúc đã thiết kế, đề tài sử dụng một bộ các công cụ mã nguồn mở được lựa chọn kỹ lưỡng, đảm bảo tính linh hoạt, hiệu quả về chi phí và khả năng mở rộng, được tóm tắt trong bảng dưới đây:

Lớp	Công cụ	Vai trò chính
Thu thập	Filebeat, Auditbeat	Agent gọn nhẹ, thu thập log file và log kiểm toán (audit) từ máy chủ Linux.
Xử lý	Logstash	Trung tâm xử lý, thực hiện parsing, filtering và enrichment (GeoIP).
Lưu trữ & Tìm kiếm	Elasticsearch	Lưu trữ, lập chỉ mục và cung cấp khả năng tìm kiếm, phân tích dữ liệu log.
Trực quan hóa	Kibana	Cung cấp giao diện để tạo dashboard, biểu đồ và điều tra dữ liệu.
Phát hiện & Cảnh báo	ElastAlert	Framework mã nguồn mở để thực thi các quy tắc phát hiện và gửi cảnh báo.
Quản lý sự cố	React, Node.js, MongoDB	Xây dựng giao diện Case UI và backend API để quản lý vòng đời sự cố.
Hạ tầng	Docker, Docker Compose	Container hóa các thành phần, giúp triển khai và quản lý một cách nhất quán.

4.1. Quy trình và các bước triển khai chi tiết

Hệ thống được triển khai dựa trên công nghệ container hóa với Docker và Docker Compose. Quy trình triển khai được chia thành các bước logic như sau:

Bước 1: Cài đặt và cấu hình môi trường

- Công cụ: Docker, Docker Compose.
- Thực hiện:
    - Cài đặt Docker Engine và Docker Compose trên máy chủ triển khai (Ubuntu Server).
    - Tạo cấu trúc thư mục dự án, bao gồm các thư mục con cho từng dịch vụ để chứa các file cấu hình tương ứng.
    - Tạo file .env để quản lý tập trung các biến môi trường, mật khẩu và các thông tin nhạy cảm.

Bước 2: Triển khai Lớp Lưu trữ, Xử lý và Trực quan hóa (ELK Stack)

- Công cụ: Elasticsearch, Logstash, Kibana.
- Thực hiện:
    - Elasticsearch: Cấu hình trong docker-compose.yml để khởi chạy một node Elasticsearch, thiết lập các tham số về bộ nhớ (heap size) và bật xác thực người dùng, mã hóa TLS.
    - Logstash: Tạo file pipeline logstash.conf để định nghĩa luồng xử lý từ input (Beats), filter (grok, geoip) đến output (Elasticsearch).
    - Kibana: Cấu hình để kết nối và xác thực với Elasticsearch, cho phép người dùng truy cập giao diện trực quan hóa.

Bước 3: Triển khai Lớp Phát hiện và Cảnh báo (ElastAlert)

- Công cụ: ElastAlert.
- Thực hiện:
    - Cấu hình file config.yaml chính để ElastAlert kết nối đến Elasticsearch.
    - Tạo các file quy tắc (.yaml) cho từng kịch bản tấn công (Brute-force, Lạm dụng đặc quyền,...) trong thư mục rules/. Mỗi quy tắc định nghĩa rõ logic truy vấn, ngưỡng kích hoạt và hành động cảnh báo (webhook).
    - Sơ đồ dưới đây minh họa luồng xử lý tự động khi một cảnh báo được kích hoạt:

Bước 4: Triển khai Lớp Quản lý Sự cố (Case Management UI)

- Công cụ: React (Frontend), Node.js (Backend), MongoDB (Database).
- Thực hiện:

- Backend (Node.js): Xây dựng các API endpoint để quản lý vòng đời sự cố. Tạo một endpoint đặc biệt để nhận webhook từ ElastAlert và tự động tạo sự cố.
- Frontend (React): Xây dựng giao diện người dùng cho phép hiển thị, tương tác, cập nhật và ghi nhận dòng thời gian điều tra sự cố.
- Database (MongoDB): Khởi tạo database và các collection cần thiết để lưu trữ dữ liệu về sự cố.

4.2. Phương pháp và Kịch bản kiểm thử

Để xác minh tính đúng đắn của hệ thống, các phương pháp kiểm thử từ tích hợp đến toàn diện (end-to-end) đã được áp dụng.

4.2.1. Kiểm thử tích hợp (Integration Testing)

Mục tiêu: Đảm bảo luồng dữ liệu từ nguồn đến đích được thông suốt.

Kịch bản:

- Beats → Logstash: Trên máy chủ nguồn, tạo một dòng log mới trong file auth.log. Kiểm tra log của container Logstash để xác nhận đã nhận được sự kiện.
- Logstash → Elasticsearch: Sau khi Logstash xử lý, sử dụng giao diện Dev Tools của Kibana để truy vấn và xác nhận sự kiện đã được lưu trữ trong Elasticsearch với đầy đủ các trường dữ liệu đã được làm giàu (ví dụ: geoip.country_name).

4.2.2. Kiểm thử toàn diện (End-to-End Testing)

Mục tiêu: Kiểm tra toàn bộ quy trình từ khi tấn công xảy ra cho đến khi sự cố được ghi nhận và cảnh báo được gửi đi.

Kịch bản 1: Giả lập tấn công Brute-force

- Thực hiện: Sử dụng công cụ hydra để tạo ra 30 lần đăng nhập SSH thất bại vào một máy chủ được giám sát trong vòng 5 phút.
- Kết quả mong đợi: (1) Nhận được cảnh báo mức độ "High" qua kênh liên lạc. (2) Một sự cố mới được tự động tạo trong Case Management UI. (3) Dashboard trên Kibana hiển thị một đỉnh nhọn (spike) trong biểu đồ đăng nhập thất bại.

Kịch bản 2: Giả lập Lạm dụng đặc quyền

- Thực hiện: Đăng nhập vào máy chủ được giám sát và thực thi lệnh sudo chmod 777 /etc/passwd.

- Kết quả mong đợi: (1) Gần như ngay lập tức, nhận được cảnh báo mức độ "Critical". (2) Cảnh báo chứa thông tin về người dùng, câu lệnh và tệp tin bị ảnh hưởng. (3) Sự cố tương ứng được tạo trong Case Management UI.

Kịch bản 3: Giả lập Đăng nhập từ vị trí đáng ngờ

- Thực hiện: Gửi một sự kiện log giả lập trực tiếp đến Logstash, mô phỏng một lần đăng nhập thành công (outcome: "success") từ một địa chỉ IP thuộc một quốc gia lạ.
- Kết quả mong đợi: (1) Nhận được cảnh báo mức độ "Medium" về việc có đăng nhập từ vị trí địa lý mới. (2) Sự cố được tạo trong Case Management UI, cho phép chuyên viên bắt đầu quy trình xác minh.

4.2.3. Kịch bản Lạm dụng đặc quyền

- Mục tiêu: Phát hiện các thay đổi trái phép trên các file hệ thống quan trọng.
- Logic:
    1. Sử dụng Auditbeat để theo dõi các hành vi thay đổi file tại các đường dẫn được bảo vệ (ví dụ: /etc/passwd, /etc/sudoers).
    2. Lọc các sự kiện có event.action là updated hoặc created trên các đường dẫn này.
    3. Kích hoạt cảnh báo với mức độ nghiêm trọng Critical ngay lập tức.
- Payload cảnh báo: Cảnh báo sẽ bao gồm thông tin về người dùng thực hiện, tiến trình, và file đã bị thay đổi.

4.3. Triển khai hạ tầng và các thành phần

Toàn bộ hệ thống được triển khai bằng Docker và quản lý thông qua file docker-compose.yml, đảm bảo tính nhất quán và dễ dàng tái tạo môi trường.

1. Cấu hình lớp thu thập dữ liệu (Beats): Các agent thu thập được cài đặt trên máy chủ nguồn, bao gồm Filebeat để thu thập nhật ký hệ thống và các sự kiện xác thực, cùng với Auditbeat để giám sát các hoạt động liên quan đến tính toàn vẹn tệp tin và thực thi lệnh. Cả hai agent đều được định cấu hình để chuyển tiếp dữ liệu một cách an toàn đến Logstash qua kết nối được mã hóa bằng giao thức TLS..
2. Cấu hình Logstash: Một pipeline được thiết lập để nhận dữ liệu từ Beats. Các bộ lọc (filters) được sử dụng để:
    - Phân tích log bằng grok để trích xuất các trường.

- Thêm thông tin vị trí địa lý bằng geoip.
- Chuẩn hóa các trường dữ liệu.
- Gửi dữ liệu đã xử lý đến Elasticsearch.

3. Cấu hình Elasticsearch & Kibana:
    - Một Index Template được tạo để đảm bảo các log mới được lập chỉ mục với đúng kiểu dữ liệu (mapping).
    - Chính sách ILM (Index Lifecycle Management) được thiết lập để tự động chuyển dữ liệu cũ từ tầng "hot" (truy cập nhanh) sang "warm" và cuối cùng là xóa đi sau một khoảng thời gian nhất định (ví dụ: 90 ngày) để tiết kiệm không gian lưu trữ.
4. Cấu hình ElastAlert: Các file quy tắc (dạng YAML) cho từng kịch bản phát hiện được tạo và ánh xạ đến các kênh cảnh báo tương ứng (Email, Slack).
5. Triển khai Case UI: Ứng dụng Node.js và React được container hóa và kết nối với cơ sở dữ liệu MongoDB để lưu trữ thông tin các sự cố.

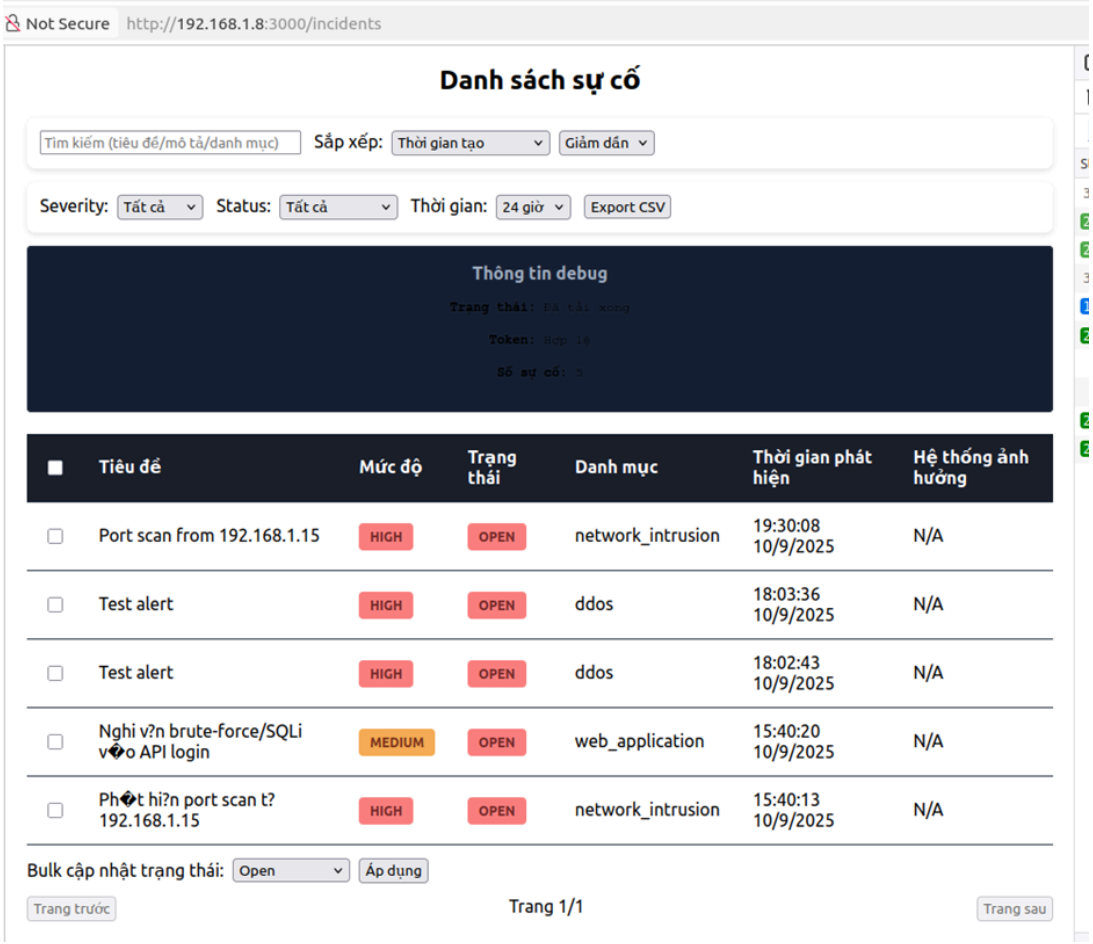
CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ

Chương này trình bày các kết quả thực tế mà hệ thống đã đạt được sau khi triển khai, đồng thời đưa ra những đánh giá khách quan về các ưu điểm và hạn chế của giải pháp.

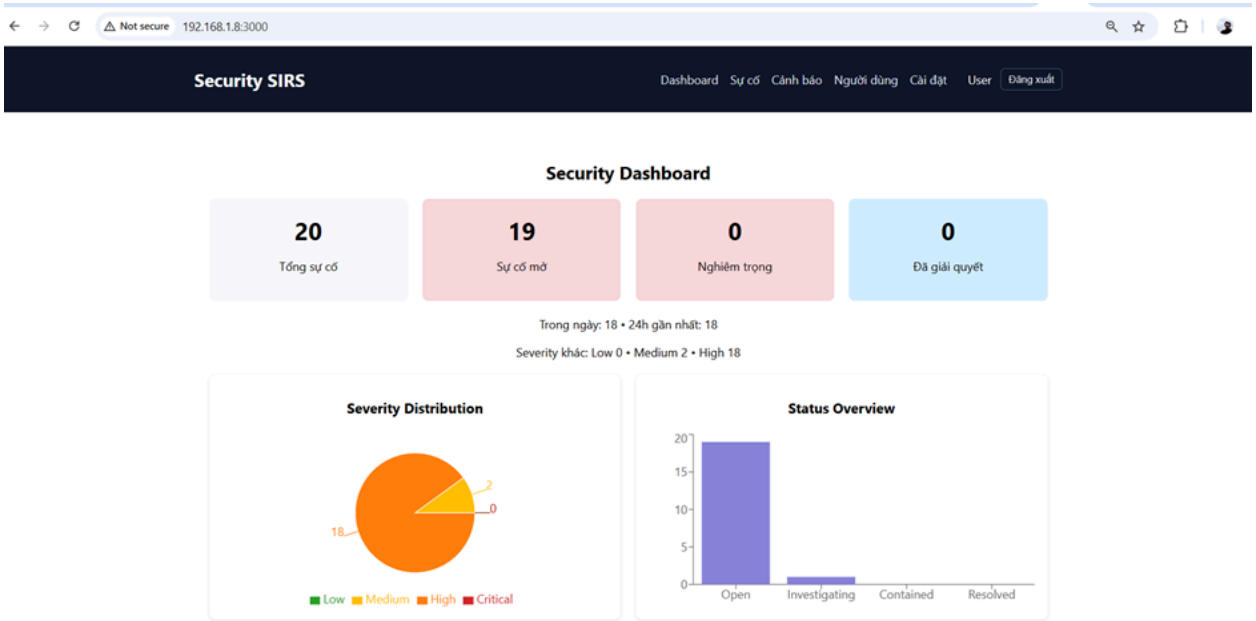
5.1. Kết quả đạt được

Hệ thống đã được triển khai thành công và cho thấy khả năng đáp ứng các yêu cầu đã đề ra. Các kết quả chính được thể hiện qua các giao diện giám sát, cảnh báo mẫu và quy trình xử lý sự cố.

5.1.1. Giao diện giám sát và cảnh báo mẫu



Hình 5: Ví dụ về các biểu đồ phân tích chi tiết trong một bảng điều khiển



Hình 6: Giao diện Bảng điều khiển chính của Hệ thống Quản lý Sự cố (Case Management UI)

- Dashboard Giám sát Xác thực (Authentication Dashboard): Giao diện này cung cấp một cái nhìn tổng quan về các hoạt động đăng nhập trên toàn hệ thống. Chuyên viên phân tích có thể nhanh chóng xác định các xu hướng bất thường thông qua các biểu đồ về số lần đăng nhập thành công/thất bại theo thời gian, danh sách các người dùng và địa chỉ IP có hoạt động đáng ngờ nhất, và một bản đồ địa lý hiển thị vị trí các lần đăng nhập.
- Dashboard Giám sát Kiểm toán (Audit Dashboard): Giao diện này tập trung vào việc theo dõi các thay đổi trên hệ thống và các hành vi sử dụng đặc quyền. Nó cho phép theo dõi các sự kiện thay đổi file trên các đường dẫn quan trọng và các lệnh sudo được thực thi, giúp nhanh chóng phát hiện các hành vi lạm dụng đặc quyền.
- Cảnh báo mẫu: Hệ thống đã chứng minh khả năng gửi cảnh báo thời gian thực qua các kênh như Email và Slack. Các cảnh báo được chuẩn hóa với đầy đủ thông tin ngữ cảnh, bao gồm:
- Tên quy tắc phát hiện và mức độ nghiêm trọng.
- Thực thể bị ảnh hưởng (ví dụ: user "alice").
- Tóm tắt các thông tin chính (ví dụ: "68 lần đăng nhập thất bại từ 12 địa chỉ IP").
- Liên kết "một chạm" (deep link) trỏ thẳng đến trang Discover trên Kibana với các bộ lọc đã được áp dụng sẵn, giúp rút ngắn tối đa thời gian điều tra.

5.1.2. Kết quả phát hiện và ví dụ về dòng thời gian sự cố

Hệ thống đã được thử nghiệm với các kịch bản tấn công giả lập và cho thấy kết quả phát hiện chính xác:

- Kịch bản Brute-force: Khi một kịch bản tấn công Brute-force được thực thi, hệ thống đã gửi cảnh báo mức độ High. Chuyên viên phân tích, dựa vào thông tin từ cảnh báo, đã nhanh chóng truy cập vào Case UI, tạo một sự cố mới và ghi nhận các hành động xử lý như "Tạm khóa tài khoản người dùng" và "Chặn các địa chỉ IP nguồn trên tường lửa". Toàn bộ quá trình này được ghi lại một cách minh bạch trên dòng thời gian của sự cố.
- Kịch bản Đăng nhập đáng ngờ: Một lần đăng nhập thành công từ một quốc gia mới đã kích hoạt một cảnh báo mức độ Medium. Dòng thời gian sự cố cho thấy các bước xử lý như "Liên hệ người dùng để xác nhận" và "Yêu cầu người dùng đổi mật khẩu", trước khi đóng sự cố.

5.2. Đánh giá hệ thống

5.2.1. Ưu điểm

- Hiệu quả về chi phí: Việc sử dụng hoàn toàn các công cụ mã nguồn mở (ELK Stack, ElastAlert, Docker) giúp loại bỏ chi phí bản quyền, làm cho giải pháp rất phù hợp với các tổ chức có ngân sách hạn chế.
- Tính linh hoạt và khả năng tùy chỉnh cao: Kiến trúc mã nguồn mở cho phép tùy chỉnh sâu từ quy trình xử lý log, nội dung cảnh báo cho đến giao diện người dùng, đáp ứng các nhu cầu đặc thù của từng tổ chức.
- Tự động hóa và rút ngắn thời gian phản ứng: Hệ thống tự động hóa phần lớn quy trình từ thu thập, phân tích đến cảnh báo. Các cảnh báo giàu ngữ cảnh và các liên kết "một chạm" giúp giảm đáng kể thời gian từ lúc phát hiện đến lúc có bằng chứng (Time-to-Evidence), qua đó rút ngắn MTTR.
- Tăng cường khả năng kiểm toán: Giao diện quản lý sự cố (Case UI) với tính năng dòng thời gian (timeline) đảm bảo mọi hành động xử lý đều được ghi lại, tạo ra một hồ sơ sự cố minh bạch và đầy đủ, phục vụ tốt cho việc kiểm toán và rút kinh nghiệm.

5.2.2. Hạn chế

- Đòi hỏi kỹ năng vận hành: Việc triển khai và vận hành hiệu quả một hệ thống dựa trên ELK Stack đòi hỏi đội ngũ kỹ thuật phải có kiến thức chuyên sâu về Elasticsearch, Logstash và quản trị hệ thống Linux.
- Phạm vi quy tắc phát hiện còn hạn chế: Bộ quy tắc hiện tại mới chỉ tập trung vào 3 kịch bản cơ bản. Để đối phó với các mối đe dọa đa dạng hơn, hệ thống cần được mở rộng với nhiều quy tắc phức tạp hơn, có thể ánh xạ theo các framework như MITRE ATT&CK.
- Thiếu tích hợp nâng cao: Hệ thống hiện chưa tích hợp với các nguồn tri thức về mối đe dọa (Threat Intelligence) để làm giàu dữ liệu, cũng như chưa có khả năng phản ứng tự động (SOAR - Security Orchestration, Automation and Response).
- Tính sẵn sàng cao (High Availability) chưa được triển khai: Trong phiên bản hiện tại, các thành phần như Elasticsearch và Logstash được triển khai dưới dạng các node đơn. Để đảm bảo hệ thống hoạt động liên tục trong môi trường production, cần phải triển khai chúng dưới dạng cụm (cluster) có tính sẵn sàng cao.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết lại những kết quả cốt lõi mà đề tài đã đạt được, khẳng định giá trị của giải pháp và đề xuất các hướng phát triển tiềm năng để hoàn thiện và nâng cao năng lực của hệ thống trong tương lai.

6.1. Kết luận

Đề tài đã trình bày và triển khai thành công một nền tảng Bảng điều khiển Giám sát và Phản ứng Sự cố An ninh mạng toàn diện dựa trên bộ công cụ mã nguồn mở ELK Stack. Giải pháp đã hợp nhất thành công chuỗi quy trình an ninh từ thu thập → chuẩn hóa → lưu trữ → trực quan hóa → phát hiện → cảnh báo → phản ứng theo thời gian gần thực.

Những đóng góp và kết quả chính của đề tài bao gồm:

Hiện thực hóa một quy trình an ninh đầu cuối: Hệ thống đã chứng minh khả năng thu thập log ổn định, chuẩn hóa và làm giàu dữ liệu một cách nhất quán, đồng thời cung cấp các công cụ trực quan mạnh mẽ và cơ chế cảnh báo tự động, giúp rút ngắn đáng kể Thời gian Trung bình để Phát hiện (MTTD) và Phản ứng (MTTR).

Xây dựng nền tảng cho việc kiểm toán: Với giao diện Quản lý sự cố (Case UI) và tính năng dòng thời gian (timeline), mọi bước trong quá trình điều tra và xử lý đều được ghi

lại một cách minh bạch, đáp ứng các yêu cầu khắt khe về kiểm toán và giúp các tổ chức rút kinh nghiệm sau mỗi sự cố.

Khẳng định tính khả thi của giải pháp mã nguồn mở: Đề tài đã cho thấy việc xây dựng một hệ thống giám sát an ninh hiệu quả, linh hoạt và có khả năng mở rộng là hoàn toàn khả thi mà không cần phụ thuộc vào các giải pháp thương mại đắt đỏ.

Về tổng thể, giải pháp không chỉ là một công cụ kỹ thuật mà còn là một phương pháp luận, đặt nền móng vững chắc cho các tổ chức trong việc xây dựng và vận hành một trung tâm giám sát an ninh (SOC) hiện đại và hiệu quả.

6.2. Hướng phát triển trong tương lai

Dựa trên những hạn chế đã được phân tích ở chương trước, hệ thống có thể được cải tiến và mở rộng theo các hướng sau để nâng cao năng lực:

- Mở rộng và làm giàu bộ quy tắc phát hiện:
    - Phát triển thêm các quy tắc phức tạp hơn, bao phủ nhiều kỹ thuật tấn công khác nhau theo chuẩn MITRE ATT&CK.
    - Tích hợp với các nguồn Tri thức về Mối đe dọa (Threat Intelligence) từ bên ngoài để làm giàu dữ liệu, ví dụ như danh sách các địa chỉ IP độc hại, giúp tăng độ chính xác của cảnh báo và giảm thiểu các cảnh báo sai (false positives).
- Tích hợp khả năng tự động hóa phản ứng (SOAR):
    - Kết nối hệ thống với một nền tảng Điều phối, Tự động hóa và Phản ứng An ninh (SOAR).
    - Xây dựng các kịch bản (playbook) phản ứng tự động cho các loại cảnh báo phổ biến, ví dụ: tự động chặn một địa chỉ IP trên tường lửa khi phát hiện tấn công Brute-force, hoặc tự động vô hiệu hóa một tài khoản người dùng khi có đăng nhập đáng ngờ.
- Nâng cao năng lực phân tích và khả năng mở rộng:
    - Triển khai các thành phần Elasticsearch và Logstash dưới dạng cụm (cluster) nhiều node để đảm bảo tính sẵn sàng cao (High Availability) và khả năng chịu tải tốt hơn.
    - Ứng dụng các thuật toán Machine Learning (có sẵn trong ELK Stack) để xây dựng mô hình hành vi người dùng thông thường, từ đó phát hiện các điểm bất thường (anomaly detection) mà các quy tắc tĩnh có thể bỏ sót.

- Hoàn thiện quy trình vận hành và quản lý:
    - Triển khai giải pháp Đăng nhập một lần (Single Sign-On - SSO) để đơn giản hóa việc truy cập và quản lý người dùng.
    - Xây dựng các bộ runbook (tài liệu hướng dẫn vận hành) chi tiết và tổ chức các buổi đào tạo cho đội ngũ chuyên viên phân tích, đảm bảo họ có thể khai thác tối đa sức mạnh của hệ thống.